

关于LSTM：起源 本身 延伸

关于LSTM：起源 本身 延伸

#Silviu·Pitis

#机器学习

#LSTM

#RNN

#Mr_RunHorse

#手绘风原理图

#没翻译完

引言

初次接触长短期记忆网络（LSTM）时，我很难忽略它的复杂性。我不理解它为何要这样设计，只知道它效果很好。事实证明，LSTM 是可以被理解的；尽管表面看起来复杂，LSTM 实际上基于几条极其简单、甚至很优美的神经网络洞见。这篇文章是我当初学习循环神经网络（RNN）时希望能读到的内容。

在这篇文章里，我们会做几件事：

- 我们会定义并描述通用的 RNN，重点讲**普通 vanilla RNN** 的局限，正是这些局限催生了 LSTM。
- 我们会讲解 LSTM 结构背后的直觉，借此一步步搭建并推导出 LSTM。推导过程中我们还会推出 GRU。我们也会推导出一种伪 LSTM，它在理论和效果上都优于标准 LSTM。
- 我们会把这些直觉延伸，展示它们如何直接引出近年一些令人兴奋的架构：高速公路网络、残差网络，以及神经图灵机。

这篇文章讲**理论**，不讲工程实现。如果你想了解如何用 TensorFlow 实现 RNN，可以看我另两篇文章：[《TensorFlow 中的循环神经网络（一）》](#)和[《TensorFlow 中的循环神经网络（二）》](#)。

译者引言

- 这是我翻译的，希望大家可以看懂。
- 原文 [Written Memories: Understanding, Deriving and Extending the LSTM](#)的著作权归原作者 [Silviu Pitis](#) 所有。
- 示意图均为本人（[Mr_RunHorse@163.com](#)）精心根据原文配图修改创作
- 几种神经网络的对比图和公式比较为译者本人后加的，希望便于理解

版权声明

本译文仅用于非商业学习交流，不代表原作者观点。文中所有示意图为译者原创，转载请注明出处。

目录 / 快速跳转

- 循环神经网络
 - RNN 能做什么；如何选择时间步
 - 普通 RNN
 - 信息畸变与梯度消失/梯度爆炸
 - 梯度消失的充分数学条件
 - 避免梯度消失的最小权重初始化
 - 时间反向传播与梯度消失
 - 应对梯度消失/爆炸
 - 书写式记忆：LSTM 背后的直觉
 - 用选择性控制与协调写入
 - 门作为选择性机制
 - 把门拼接起来，推导出原型 LSTM
 - 三种可用模型：归一化原型、GRU、伪 LSTM
 - 推导 LSTM
-

以下的我没精力再ai翻译了

- 带窥孔的 LSTM
 - 基础 LSTM 与伪 LSTM 的实验对比
 - 扩展 LSTM
-

前置知识

这篇文章假设读者已经熟悉：

- 前馈神经网络
- 反向传播
- 基础线性代数

除此之外的内容我们都会从头回顾，首先从通用 RNN 开始。

一、循环神经网络（Recurrent neural networks）

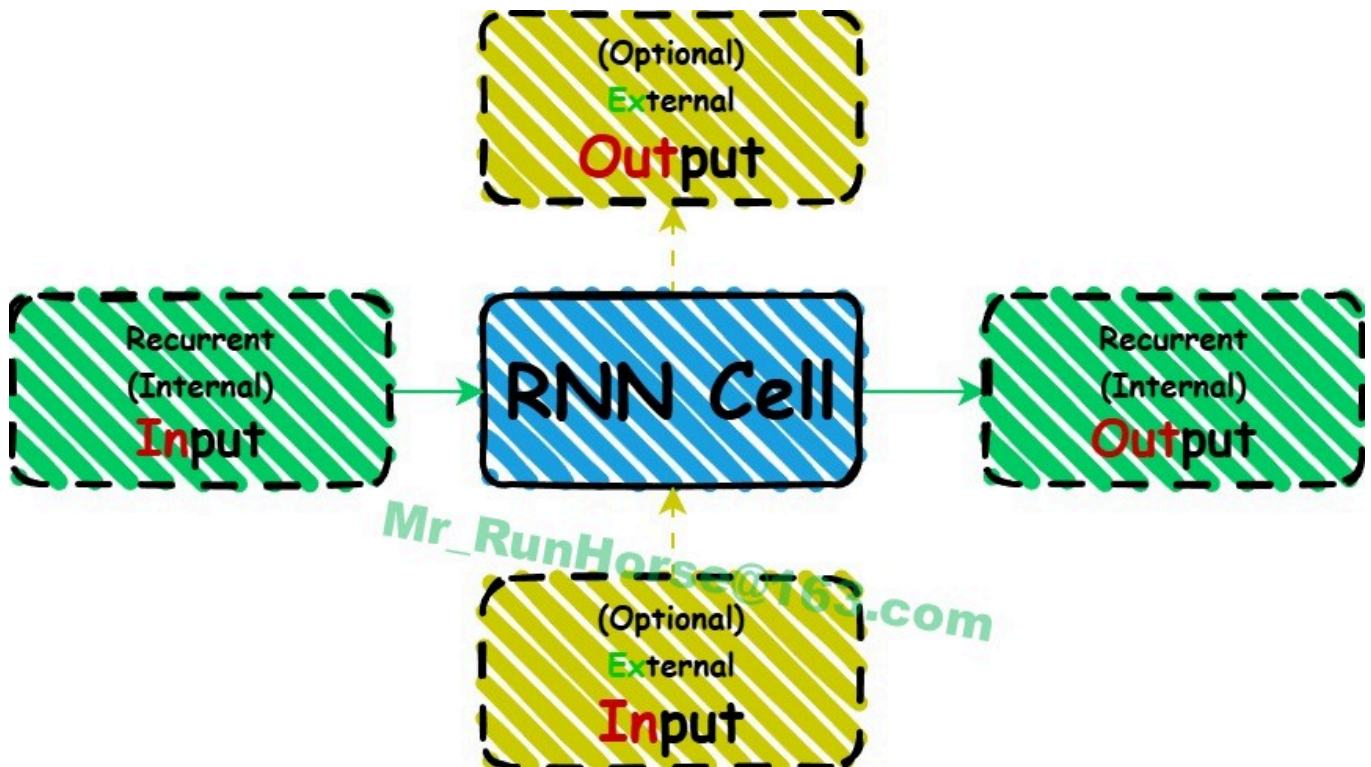
从一个时刻到下一个时刻，我们的大脑像一个函数：它接收来自感官的外部输入和来自思维的内部输入，并以动作（外部）和新想法（内部）的形式产生输出。我们看到一只熊，然后想到

“熊”。我们可以用前馈神经网络模拟这种行为：我们可以训练一个前馈网络，在看到熊的图片时输出“熊”。

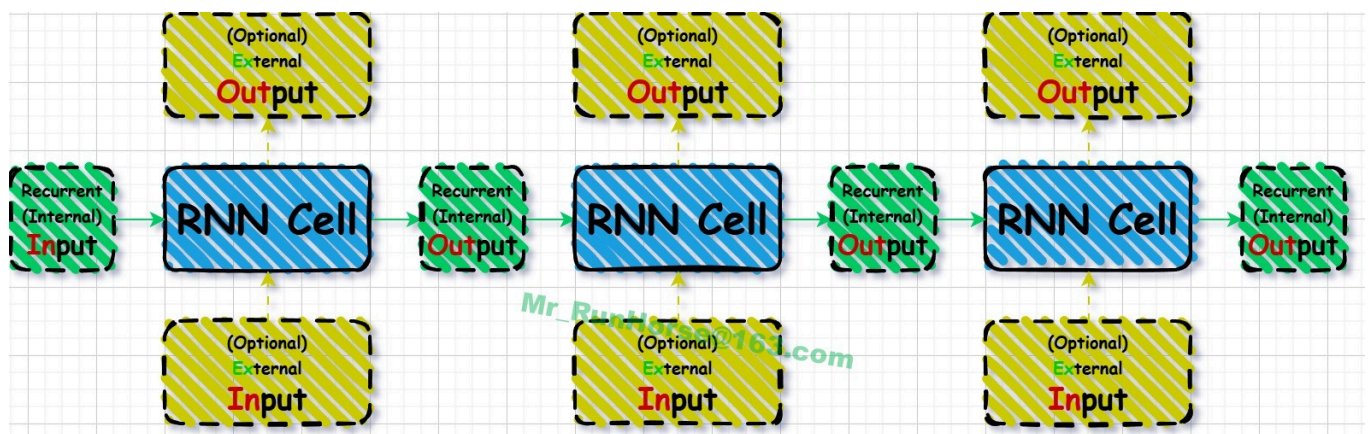
但大脑不是一次性函数。它在时间上不断重复运行。我们看到熊→想到“熊”→想到“跑”。重要的是，把图片转换成“熊”这个想法的**同一个函数**，也把“熊”这个想法转换成“跑”这个想法。它是一个**循环函数**，可以用循环神经网络（RNN）来建模。

RNN 是**相同前馈网络的组合**，每个时刻/时间步对应一个，我们称之为“RNN 单元”。注意，这是比通常定义更宽泛的 RNN 定义（普通 RNN 会在后面作为 LSTM 的前身讲解）。这些单元会作用于自己的输出，因此可以组合。它们也可以接收外部输入、产生外部输出。

下面是单个 RNN 单元的示意图：



下面是三个组合在一起的 RNN 单元：



你可以把循环输出看作传递到下一个时间步的**状态**。因此，一个 RNN 单元接收上一时刻状态和（可选）**当前输入**，并产生**当前状态**和（可选）**当前输出**。

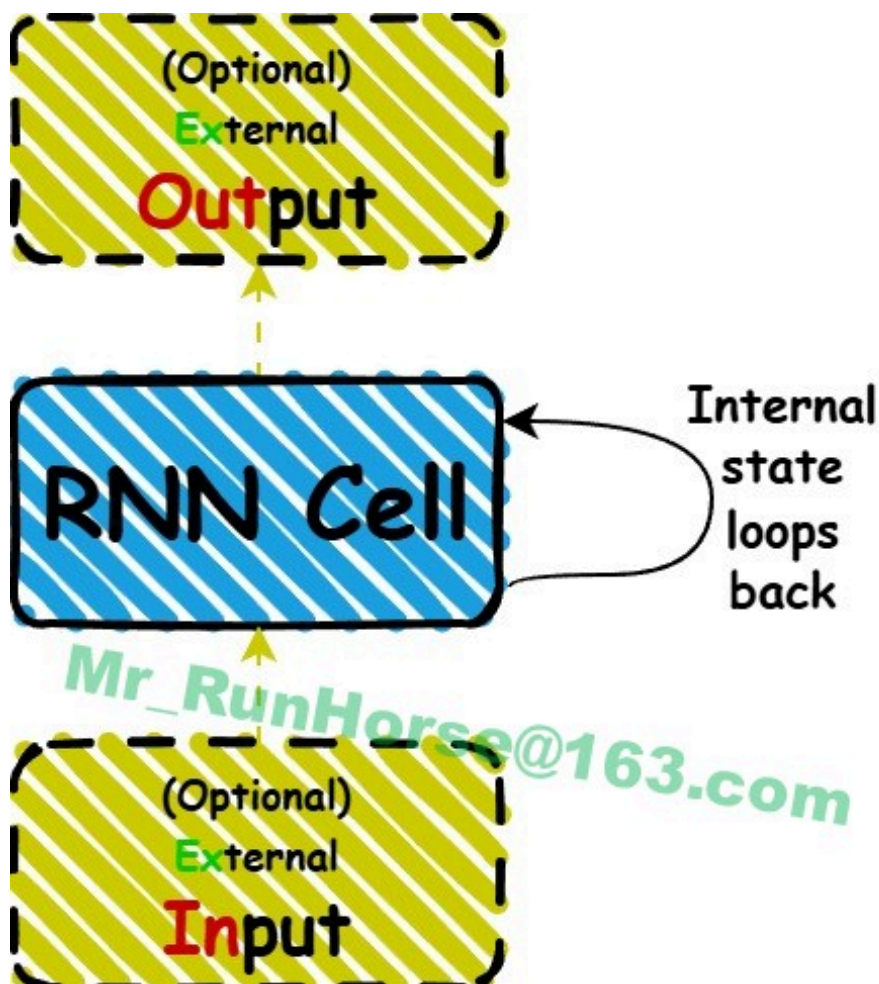
RNN 单元的代数描述：

$$\begin{pmatrix} s_t \\ o_t \end{pmatrix} = f \left(\begin{pmatrix} s_{t-1} \\ x_t \end{pmatrix} \right)$$

其中：

- s_t 、 s_{t-1} ：当前与上一时刻状态
- o_t ：当前输出（可为空）
- x_t ：当前输入（可为空）
- f ：循环函数

大脑是这么运行的：当前神经活动覆盖过去的神经活动。RNN 也可以看作原地运行：因为所有 RNN 单元完全相同，它们可以被视为同一个对象，RNN 单元的“状态”在每个时间步被覆盖。这就是循环示意图：



大多数 RNN 介绍都从这种“单单元循环”视角开始，但我认为**时序展开视角**更直观，尤其在思考反向传播时。从单单元循环出发，把 RNN“展开”就得到上面的时序结构。

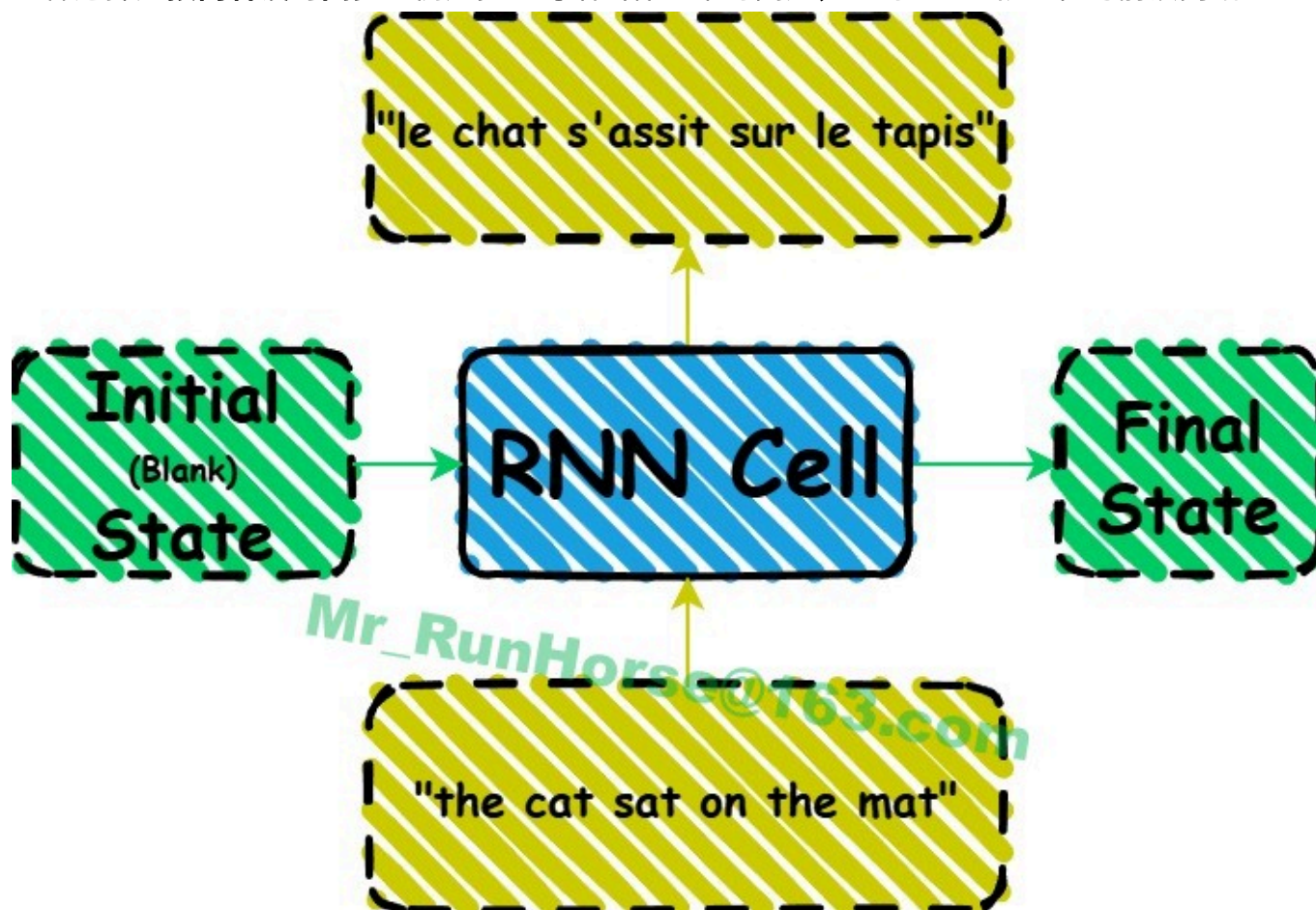
RNN 能做什么；如何选择时间步

上面描述的 RNN 结构非常通用。理论上它可以做任何事：如果每个单元内部的神经网络至少有一个隐藏层，每个单元就成为**通用函数逼近器**。这意味着 RNN 单元可以模拟任何函数，进而 RNN 在理论上可以完美模拟大脑。虽然我们不知道大脑理论上可以这样建模，但真正设计并训练出能做到这一点的 RNN 完全是另一回事。不过我们确实正在取得不错的进展。

以大脑作类比，想知道如何用 RNN 处理任务，只需要问：**人类会怎么处理同样的任务。**

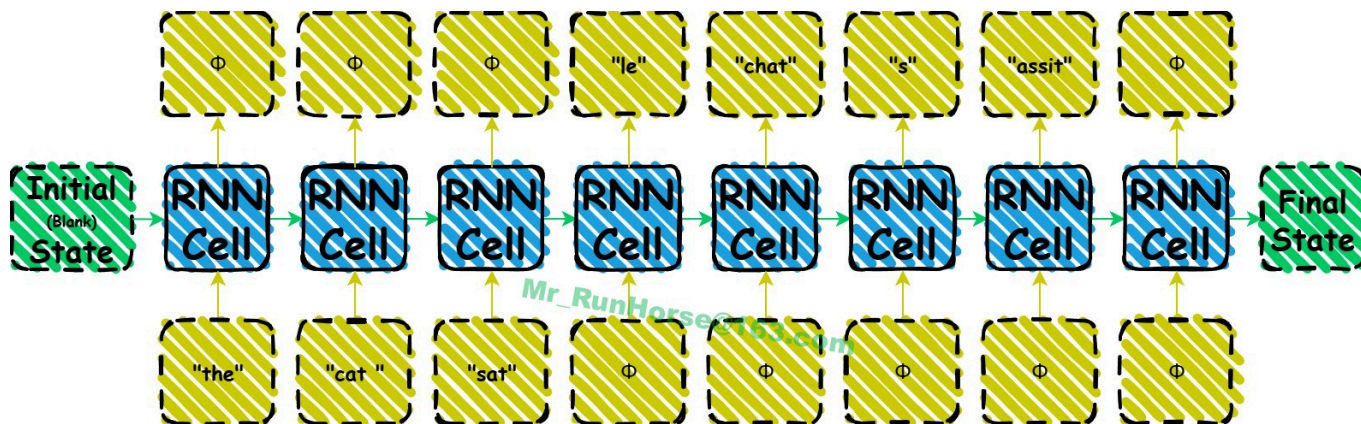
以英译法为例：人读一句英文（the cat sat on the mat），停顿，然后写出法语译文（le chat s'assit sur le tapis）。用 RNN 模拟这一行为，除了设计 RNN 单元本身（暂时当作黑盒），我们唯一需要做的选择是**时间步如何划分**，这决定了输入输出的形式，即 RNN 如何与外部世界交互。

一种选择是**按内容设时间步**。例如把整句话当作一个时间步，此时 RNN 就退化为前馈网络：



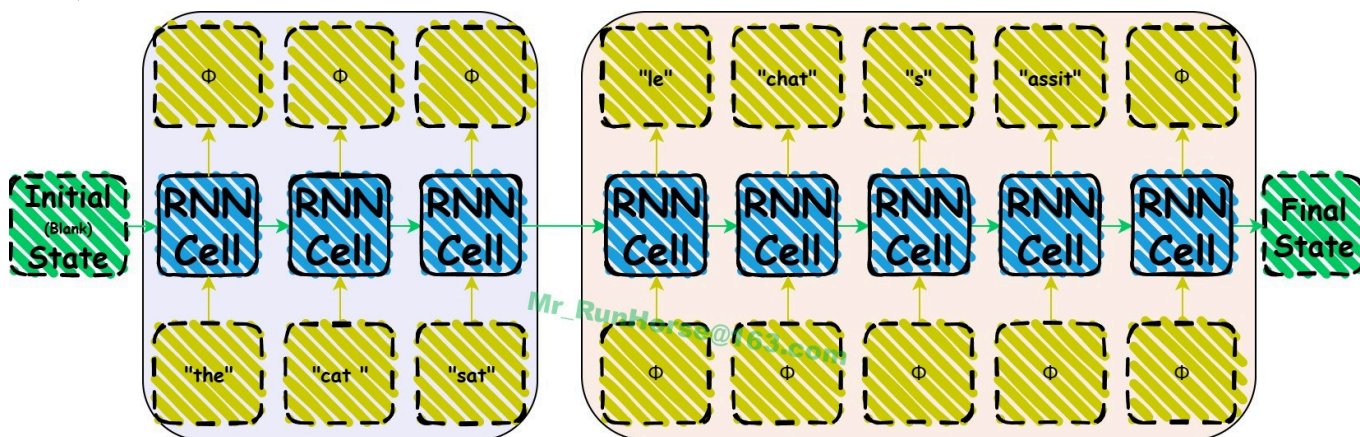
翻译单句时最终状态不重要；但如果句子是段落的一部分，最终状态会包含前文信息，就很重要。注意初始状态标为空白，但在单独评估序列时，把初始状态训练成可学习变量往往更有用。最优的“序列开始”状态表示可能不是全零。

另一种选择是**每个词/每个字符作为一个时间步**。下图展示 RNN 逐词翻译“the cat sat”的过程：



第一个时间步后，状态包含“the”的内部表示；第二个时间步后包含“the cat”；第三个后包含“the cat sat”。网络在前三个时间步不输出，收到空白输入时才开始输出，表示输入结束；输出完成时产生空白信号表示结束。

实际中，即使是深度 LSTM 这类强大的 RNN 结构，在同时处理多个任务（先读再译）时也可能表现不佳。为此我们可把网络拆成多个 RNN，各自专攻一个任务。本例中用一个**编码器网络读英语**，另一个**解码器网络生成法语**。



此外，解码器会把上一个真实值（训练时的目标、测试时的上一步输出）当作输入。这类 RNN 编码器-解码器模型可参考 [Cho et al. \(2014\)](#)。

注意，两个独立网络仍然符合单一 RNN 的定义：我们可以把循环函数定义为一个分岔函数，除了其他输入外，再接收一个信号，指定使用函数的哪一部分。

时间步不必基于内容，也可以是**真实时间单位**。例如把时间步设为 1 秒，强制每秒读 5 个字符，那么前三个时间步的输入可能是：the c、at sa、t on。

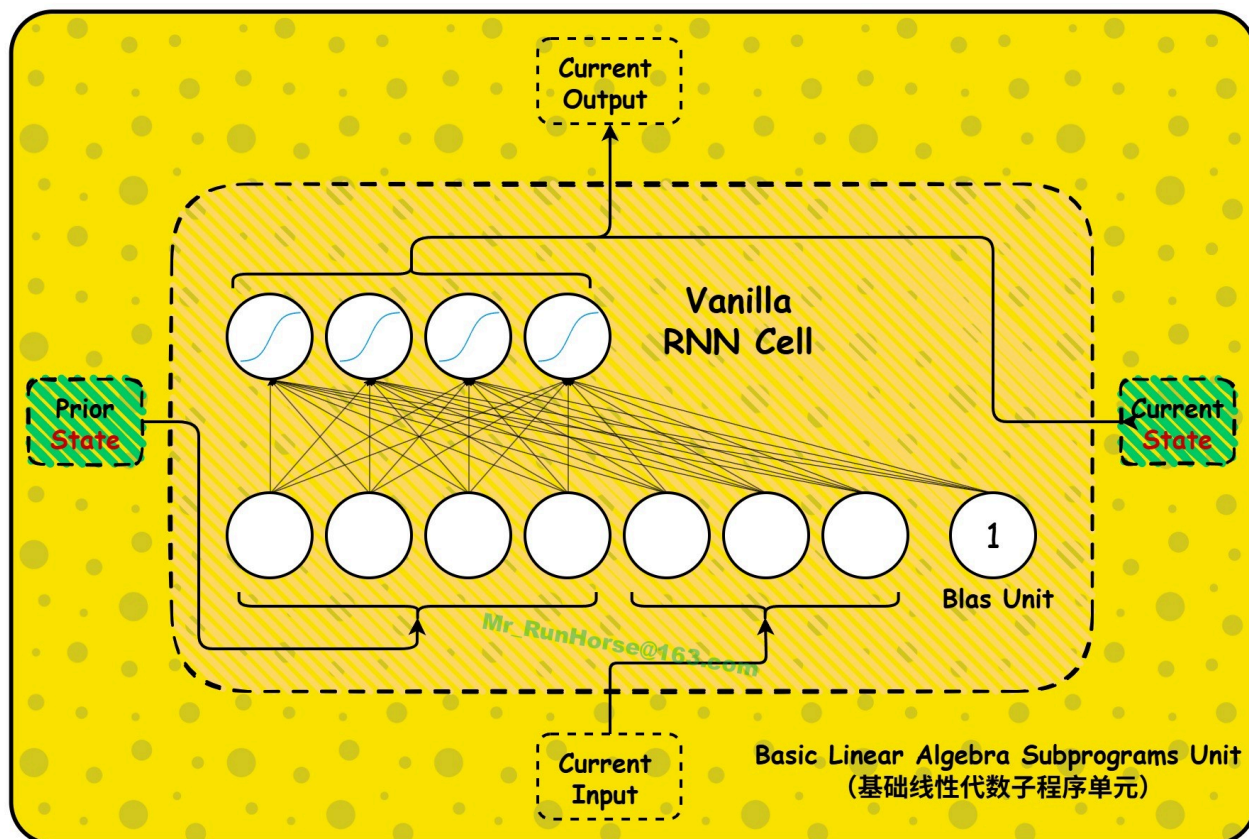
我们还可以更灵活：**让 RNN 自己决定何时推进、用什么输入**。这类似人类在翻译时会在某些词上停留更久，或回头重读。基于注意力的翻译模型就是这个思路：每一步接收整个英文序列，RNN 单元决定当前法文词最相关的部分。

这个**英译法**例子没有特殊之处。任何人类任务，我们都可以通过选择不同时间步构建不同 RNN 模型。我们甚至可以把手写数字识别这类通常用一次性函数（单时间步）的任务，重构成多时间

步任务。前馈网络做不到，RNN 可以。

普通 RNN (The vanilla RNN)

现在来看 RNN 单元的内部。最基础的 RNN 单元是**单层神经网络**，其输出同时作为当前外部输出和当前状态：



注意上一状态向量与当前状态向量长度相同。如前所述，这对 RNN 单元的组合至关重要。

普通 RNN 单元的代数描述：

$$s_t = \phi(Ws_{t-1} + Ux_t + b)$$

其中：

ϕ ：激活函数 (sigmoid、tanh、ReLU 等)

$s_t \in \mathbb{R}^n$ ：当前状态 (也是输出)

$s_{t-1} \in \mathbb{R}^n$ ：上一状态

$x_t \in \mathbb{R}^m$ ：当前输入

$W \in \mathbb{R}^{n \times n}$ 、 $U \in \mathbb{R}^{m \times n}$ 、 $b \in \mathbb{R}^n$ ：权重与偏置

n 、 m ：状态维度与输入维度

即使是这个基础 RNN 单元也相当强大。虽然单单元不满足通用函数逼近条件，但已知一串普通 RNN 单元是**图灵完备**的，可以实现任何算法 ([见 Siegelmann & Sontag, 1992](#))。理论上很好，但实践中有个问题：**用反向传播训练普通 RNN 非常困难**，甚至比训练极深的前馈网络更难。原因是**信息畸变和梯度消失/爆炸**——由重复应用同一非线性函数导致。

信息畸变与梯度消失/爆炸 (Information morphing and vanishing and exploding sensitivity)

不用大脑举例，我们把整个世界建模成 RNN：从一刻到下一刻，世界的状态被一个极其复杂的循环函数“时间”所修改。现在想：今天的一个微小变化，100 年后会对世界产生什么影响？可能蝴蝶扇动翅膀最终引发地球另一端的台风。也可能今天的行为最终毫无影响。就算爱因斯坦没发现相对论？1950 年代会有影响，但可能后来有人补上，到 2000 年差异变小，2050 年趋近于 0。也可能影响波动：爱因斯坦的发现可能源于妻子对一只蝴蝶的一句评论，于是蝴蝶效应在 20 世纪爆发，然后迅速消失。

在爱因斯坦的例子中，过去的变化是新信息（相对论），而这个新信息的引入直接来自循环函数（时间流逝）。因此我们可以把**信息本身**看作一种变化，被循环函数不断扭曲，使其效果**消失、爆炸或波动**。

这说明世界（或 RNN）的状态一直在变，当前状态对过去的变化可能**极度敏感或极度不敏感**：效果会累积或消散。这是个问题，并且普遍存在于 RNN（与前馈网络）中：

1. 信息畸变

如果信息持续畸变，我们需要它时就很难正确利用。信息最可用的状态可能出现在过去。我们不仅要学会如何利用今天的信息（假设它还以原始可用形式存在），还要学会从当前状态解码原始状态（如果可能）。这导致学习困难、效果差。

很容易证明普通 RNN 中必然发生信息畸变。假设在没有外部输入时，RNN 单元能完全保留上一状态，那么：

$$F(x) = \phi(Ws_{t-1} + b)$$

会是关于 s_{t-1} 的恒等函数。但恒等函数是线性的，而 $F(x)$ 是非线性的，矛盾。因此 RNN 单元**必然**会在时间步之间扭曲状态。普通 RNN 甚至无法完成输出 $s_t = x_t$ 这种简单任务。

这就是某些文献中所说的**退化问题**的根源（如 [He et al., 2015](#)）。作者称这一问题“出乎意料”、“反直觉”，但我希望本文能说明：退化问题（即信息畸变）其实非常自然（在很多情况下甚至是可取的）。我们后面会看到，虽然信息畸变不是 LSTM 最初的设计动机，但 LSTM 的原理恰好有效解决了这个问题。事实上，He 等人所用残差网络的有效性，也来自 LSTM 的核心原理。

2. 梯度消失与梯度爆炸

我们用反向传播算法训练 RNN。而反向传播是基于梯度的算法，梯度消失/爆炸就是梯度的消失/爆炸（后者是通用说法，但我觉得“敏感度”更直观）。梯度爆炸会导致训练崩溃；梯度消失会让长期依赖难以学习，因为反向传播会过度受近期干扰影响。这让训练变得困难。

我稍后再讲时间反向传播的训练难点，先简短用数学演示普通 RNN 为何极易梯度消失，以及训练初期如何缓解。

梯度消失的充分数学条件 (A mathematically sufficient condition for vanishing sensitivity)

本节给出普通 RNN 梯度消失的**充分条件证明**。数学味较浓，可跳过细节。思路与 [Pascanu et al. \(2013\)](#) 相近，但表述更易懂。本文还借助中值定理更进一步，得到更强结论，本质上证明的是**因果消失**而非仅敏感度消失。

梯度消失/爆炸的数学分析可追溯到 90 年代初：[Bengio et al. \(1994\)](#)、Hochreiter (1991) (德文原文，相关部分总结于 [Hochreiter & Schmidhuber, 1997](#))。

设 s_t 为 t 时刻状态向量， Δv 为状态变化 Δs_t 导致的向量变化。我们要给出一个充分条件，使得 t 时刻状态变化对 $t+k$ 时刻的影响随 $k \rightarrow \infty$ 消失，即证明：

$$\lim_{k \rightarrow \infty} \frac{\Delta s_{t+k}}{\Delta s_t} = 0$$

对比之下，Pascanu et al. (2013) 在同样充分条件下证明了：

$$\lim_{k \rightarrow \infty} \frac{\partial s_{t+k}}{\partial s_t} = 0$$

从普通 RNN 定义出发：

$$s_{t+1} = \phi(z_t), \quad \text{其中} \quad z_t = W s_t + U x_{t+1} + b$$

对多元函数应用**多元均值定理**，可证存在 $c \in [z_t, z_t + \Delta z_t]$ ，使得：

$$\begin{aligned} \Delta s_{t+1} &= [\phi'(c)] \Delta z_t \\ &= [\phi'(c)] \Delta (W s_t) \\ &= [\phi'(c)] W \Delta s_t \end{aligned}$$

设 $\|A\|$ 为矩阵2-范数， $|v|$ 为欧几里得向量范数，定义：

$$\gamma = \sup_{c \in [z_t, z_t + \Delta z_t]} \|[\phi'(c)]\|$$

注：逻辑Sigmoid激活函数满足 $\gamma \leq \frac{1}{4}$ ，tanh激活函数满足 $\gamma \leq 1$ 。⁸ 对等式两侧取向量范数，推导如下：第一个不等式由矩阵2-范数定义（连续使用两次）得到，第二个不等式由上确界定义得到：

$$\begin{aligned} |\Delta s_{t+1}| &= \|[\phi'(c)] W \Delta s_t\| \\ &\leq \|[\phi'(c)]\| \|W\| \|\Delta s_t\| \\ &\leq \gamma \|W\| \|\Delta s_t\| \\ &= \|\gamma W\| \|\Delta s_t\| \end{aligned} \tag{1}$$

将该式沿 k 个时间步展开，可得 $|\Delta s_{t+k}| \leq \|\gamma W\|^k |\Delta s_t|$ ，

因此：

$$\frac{|\Delta s_{t+k}|}{|\Delta s_t|} \leq \|\gamma W\|^k$$

因此，若满足 $\|\gamma W\| < 1$ ，则 $\frac{|\Delta s_{t+k}|}{|\Delta s_t|}$ 随时间呈指数衰减，由此证明梯度消失的**充分条件**：

$$\lim_{k \rightarrow \infty} \frac{\Delta s_{t+k}}{\Delta s_t} = 0$$

收敛于0的条件：

何时满足 $\|\gamma W\| < 1$ ？即k步后累乘 $\rightarrow 0$

隐含层为 **Sigmoid** 时： $\gamma \leq 1/4 \rightarrow \|W\| < 4$ 就满足

隐含层为 **Tanh** 时： $\gamma \leq 1 \rightarrow \|W\| < 1$ 就满足

该推导直接给出**结论**：

若权重矩阵 W 初始化值**过小**，循环神经网络（RNN）会**因梯度消失**，无法有效学习初始阶段的信息。下文将拓展该分析，推导合理的权重初始化方案。

避免梯度消失的最小权重初始化（A minimum weight initialization for avoid vanishing gradients）

一种避免梯度消失的最小权重初始化

找到一种不会立即遭受此问题困扰的权重初始化方法是有益的。扩展上述分析以找到能让我们尽可能接近等式的 W 的初始化，会得出一个很好的结果。

首先，让我们假设 $\phi = \tanh$ 并取 $\gamma = 1$ ，⁹ 但你也可以同样容易地假设 $\phi = \sigma$ 并取 $\gamma = \frac{1}{4}$ 来得出不同的结果。

我们的目标是找到一个 W 的**初始化**，使得：

1. $\|\gamma W\| = 1$ 。
2. 我们在公式 (1) 中尽可能接近等式。

从第 1 点来看，既然我们取 γ 为 1，我们有 $\|W\| = 1$ 。从第 2 点，我们得出应该尝试将 W 的所有奇异值设为 1，而不仅仅是最大的那个。那么，如果 W 的所有奇异值都等于 1，这意味着 W 的每一列的范数都是 1（因为每一列是 $W e_i$ ，对于某个初等基向量 e_i ，且我们有 $|W e_i| = |e_i| = 1$ ）。这意味着对于第 j 列，我们有：

$$\sum_i w_{ij}^2 = 1$$

第 j 列中有 n 个条目，我们要从同一个随机分布中选择每一个，所以让我们为一个随机权重 w 找到一个分布，使得：

$$n\mathbb{E}(w^2) = 1$$

现在假设我们要在区间 $[-R, R]$ 上均匀初始化 w 。那么 w 的均值为 0，所以根据定义， $\mathbb{E}(w^2)$ 就是其方差 $\mathbb{V}(w)$ 。区间 $[a, b]$ 上均匀分布的方差由 $\frac{(b-a)^2}{12}$ 给出，由此我们得到 $\mathbb{V}(w) = \frac{R^2}{3}$ 。将其代入我们的方程，我们得到：

$$n \frac{R^2}{3} = 1$$

使得：

$$R = \frac{\sqrt{3}}{\sqrt{n}}$$

这表明我们应该从以下区间上的均匀分布初始化我们的权重：

$$\left[-\frac{\sqrt{3}}{\sqrt{n}}, \frac{\sqrt{3}}{\sqrt{n}} \right]$$

这是一个很好的结果，因为它是方阵权重矩阵的 Xavier-Glorot 初始化，但却是出于不同的动机。Xavier-Glorot 初始化由 [Glorot 和 Bengio \(2010\)](#) 提出，在实践中已被证明是一种有效的权重初始化方案。更一般地，Xavier-Glorot 方案适用于层中使用的 $m \times n$ 权重矩阵，该层的激活函数在原点附近的导数接近于 1（如 tanh），并指出我们应该根据以下区间的均匀分布来初始化权重：

$$\left[-\frac{\sqrt{6}}{\sqrt{m+n}}, \frac{\sqrt{6}}{\sqrt{m+n}} \right]$$

你可以轻松修改上述分析，以获得在使用逻辑 Sigmoid 函数（使用 $\gamma = \frac{1}{4}$ ）时以及根据不同的随机分布（例如，高斯分布）初始化权重时的初始化方案。

时间反向传播与梯度消失（Backpropagation through time and vanishing sensitivity）

用反向传播训练 RNN 和训练前馈网络很像。假设你已熟悉反向传播，只补充几点：

1. 我们沿时间反向传播误差

对于 RNN，我们需要将误差从当前的 RNN 单元沿着状态反向传播，穿越时间，回溯到之前的 RNN 单元。这使得 RNN 能够学习捕捉长期的时间依赖关系。

由于模型的参数在所有 RNN 单元之间是共享的（每个 RNN 单元都有完全相同的权重和偏置），我们需要分别计算每个时间步的梯度，然后将它们相加。这与我们在其他模型（如卷积神经网络）中反向传播误差到共享参数的方式类似。

2. 权重更新频率与梯度精度存在权衡

对于所有基于梯度的训练算法而言，在

- 参数更新的频率（反向传播的次数）
和
- 准确的长期梯度之间，存在着一种不可避免的权衡（Trade-off）。

我们以逐时间步更新梯度、同时将误差反向传播多步的场景为例进行说明：

1. 在时刻 t ，使用当前权重 $\mathbf{W}_t$ 计算当前输出 $\mathbf{o}_t$ 与当前状态 $\mathbf{s}_t$ 。
2. 第二步，利用 $\mathbf{o}_t$ 执行反向传播，将权重由 $\mathbf{W}_t$ 更新为 $\mathbf{W}_{t+1}$ 。
3. 第三步，在时刻 $t+1$ ，就像在步骤1用最初的 $\mathbf{W}_t$ 参与计算那样，我们用 $\mathbf{W}_{t+1}$ 和 $\mathbf{s}_t$ 去计算 $\mathbf{o}_{t+1}$ 和 $\mathbf{s}_{t+1}$ 。
4. 最后，利用 $\mathbf{o}_{t+1}$ 执行反向传播。但 $\mathbf{o}_{t+1}$ 是由 $\mathbf{s}_t$ 计算得到，而 $\mathbf{s}_t$ 基于旧权重 $\mathbf{W}_t$ （而非更新后的 $\mathbf{W}_{t+1}$ ）。

这意味着：我们在时间步 t 计算得到的权重梯度，是基于旧权重 $\mathbf{W}_t$ 评估，而非当前权重 $\mathbf{W}_{t+1}$ 。因此该梯度只是基于当前权重计算出的梯度的近似值。

若我们将误差反向传播至更远的时间步，这种近似偏差会持续累积放大。

我们可以通过减少参数更新（反向传播）的次数，得到精度更高的梯度，但代价是降低训练速度（该问题在训练初期影响尤为严重）。

这一权衡关系，与小批量梯度下降中选择批量大小的权衡逻辑高度相似：批量越大，梯度估计越精准，但梯度更新的次数越少。

我们也可以限制误差反向传播的步数，使其不超过参数更新的频率。但此时我们无法计算损失函数关于权重的完整梯度，这只是同一权衡问题的另一面，本质矛盾依然存在。

[Williams and Zipser \(1995\)](#) 威廉姆斯与齐普瑟于1995年的研究讨论了该效应，同时系统梳理了基于梯度的训练算法中各类梯度计算方案。

3. 梯度消失 + 权重共享 = 梯度流动失衡、过度受近期干扰

1. 前馈神经网络中的“懒惰学习”

- **现象：** 在深层前馈网络中，由于梯度在反向传播过程中呈**指数级衰减**，靠近输入端的**早期层**接收到的梯度远小于靠近输出端的**后期层**。
- **后果：**
 - 后期层权重更新快，早期层权重更新极慢。
 - **沟通错位：** 训练初期，早期层只能发出“粗糙的信号”。后期层为了降低误差，会迅速学会如何解读这些粗糙信号。
 - **锁定效应：** 一旦后期层适应了粗糙信号，早期层就没有动力去学习提取更精细、更复杂的特征了。它只需要微调这些“粗糙符号”即可。这就导致网络可能陷入次优解，无法发

挥深层网络的表征能力。

2. 循环神经网络中的“直接冲突”

RNN 的情况比前馈网络更糟，原因在于**权重共享**。

- **机制差异：** 在 RNN 中，同一个权重矩阵既充当“早期层”(处理序列开头)，又充当“后期层”(处理序列结尾)。
- **梯度冲突：**
 - 对于同一个权重，在处理序列早期数据时，梯度可能提示它应该**增加**（正梯度）。
 - 而在处理序列晚期数据时，梯度可能提示它应该**减少**（负梯度）。
- **结果：** 这种冲突会导致“未学习”现象——早期层学到的东西被后期层的梯度更新所抵消甚至破坏。

Hochreiter 和 Schmidhuber (1997)

“Backpropagation through time is too sensitive to recent distractions”

意为：随时间反向传播算法过于敏感，容易被序列末尾（近期）的信息干扰，从而“遗忘”或破坏对序列开头（早期）信息的处理能力。

4. 截断反向传播是合理选择

在训练过程中限制误差反向传播的步数，这一操作被称为**截断反向传播**。需要可以马上注意到：如果我们要拟合的**输入/输出序列是无限长的**，就必须**对反向传播进行截断**，否则算法会在反向传播阶段**停滞**。如果**序列是有限但非常长的**，受限于计算可行性，我们可能仍然需要**截断反向传播**。

然而，即便我们拥有一台可以瞬间完成无限时间步误差反向传播的超级计算机，前文提到的第2点也表明：由于**权重更新会导致梯度逐渐失真**，我们仍然需要**截断反向传播**。

最后，梯度消失为截断反向传播提供了又一个理由。**如果梯度消失，那么经过多步反向传播的梯度会变得非常小，对训练的影响微乎其微。**

需要注意的是，我们不仅要选择反向传播的截断频率，还要选择模型参数的更新频率。可参考我发布的[《截断反向传播的几种形式》](#)一文，对两种截断方法进行经验性对比；也可查阅[Williams and Zipser \(1995\)](#) **的相关讨论。

5. 还有前向梯度传播算法

你需要知道的一件有用的事（以防你自己想到这个思路）是：**反向传播（Backpropagation）并不是训练循环神经网络（RNN）的唯一选择**。我们不必反向传播误差，也可以**前向传播梯度分量**，从而在每个时间步计算误差对权重的梯度。这种替代算法被称为**实时循环学习（Real-Time Recurrent Learning, RTRL）**。

完整的 RTRL 计算开销过大，在实际中难以应用，其时间复杂度为 $O(n^4)$ 。相比之下，截断反向传播（Truncated Backpropagation Through Time, TBPTT）在参数更新频率与反向传递相同时，复杂度仅为 $O(n^2)$ 。

正如截断反向传播是对完整反向传播（时间复杂度为 $O(n^2L)$ ，当时间步数 L 很大时，该复杂度可能远高于 RTRL）的近似一样，RTRL 也存在一种近似版本，称为**分组 RTRL（Subgrouped RTRL）**。当分组大小固定时，它能达到与截断反向传播相同的 $O(n^2)$ 时间复杂度，但其梯度近似方式在本质上有所不同。

需要注意的是，RTRL 是一种基于梯度的算法，因此同样会受到**梯度消失与梯度爆炸问题**的困扰。你可以在 [Williams and Zipser \(1995\)](#) 的论文中了解更多关于 RTRL 的细节

RTRL 只是我想让你了解的一种可选方案，超出了本文的讨论范围；在后续内容中，我将默认使用截断反向传播来计算梯度。

应对梯度消失/爆炸（Dealing with vanishing and exploding gradients）

梯度爆炸会导致早期梯度出现 NaN 值，训练失效。简单解决方案是**梯度裁剪**，把梯度限制在最大值内（[Mikolov 2012](#)，[Pascanu et al. 2013](#)）。实践有效，能避免 NaN 值，让训练继续。

梯度消失在普通 RNN 中更难处理。前面注意到好的权重初始化至关重要，但只影响训练开始阶段。[Pascanu et al. \(2013\)](#) 提出引入正则项，强制反向误差流动保持稳定。在少数实验上有效，但难以证明它普遍适用——因为我们把自己对梯度流动的看法强加给模型。对某些任务这可能有益，但对某些任务我们可能希望梯度完全消失或增长，此时正则反而损害性能。

不过 LSTM 完全避开了这个问题。

二、LSTM 背后的直觉（Written memories: the intuition behind LSTMs）

信息在 RNN 单元间传递时，就像传话游戏，不断被扭曲，原始信息丢失。原始信息的微小变化可能对最终结果毫无影响，也可能完全改变结果。

如何保护信息的完整性？这就是 LSTM 的**核心原则**：

在现实世界里，为了保证信息不丢失，我们**把它写下来**。书写是对当前状态的增量修改：是**创造**（笔写在纸上）或**损坏**（刻在石头上）；被记录的主体本身不会被扭曲，反向传播时误差梯度保持恒定。

这正是[Hochreiter & Schmidhuber \(1997\)](#) 那篇里程碑论文提出的思路，**引出了 LSTM**。他们问：

如何让单个自连接单元上的误差保持恒定流动？

答案很简单：**避免信息畸变**。LSTM 状态的改变是通过**显式的加法或减法**写入的，因此状态的每个元素在没有外部干扰的情况下保持恒定：‘单元的激活值必须保持恒定……这将通过使用恒等函数来确保’。

LSTM核心原则：写下来

现实中要保证信息不丢，就写下来。书写是增量修改，它可以是**加法**（纸上的笔墨）也可以是**减法**（岩石上的雕刻），并且只要没有外界干扰，它就会保持不变。

在 LSTM 中，一切都被‘写’了下来，并且假设没有其他状态单元或外部输入的干扰，它会将先前的状态向前传递。

换句话说，状态变化是**增量式**的：

$$s_{t+1} = s_t + \Delta s_{t+1}.^{10}$$

Hochreiter 和 Schmidhuber 观察到，单纯的“把信息写下来”的思路之前已经有人尝试过，但效果并不理想。要理解原因，我们可以看看当我们持续写入变化时会发生什么：我们的**写入**有正有负，理论上可以互相抵消，所以状态不一定会爆炸。但事实证明，网络很难学会如何协调这些写入操作。尤其是在训练初期：我们的参数是随机初始化的，网络会进行一些完全随机的写入操作。从训练一开始，我们的“画布”就会变成这样：



即使我们最终学会了如何正确协调写入，也很难在这片混乱之上记录任何有用的信息（尽管这个例子中的混乱非常漂亮，而且还算规整，大约10年前它还[价值1.4亿美元](#)）。这就是 LSTM 要解决的根本问题：**失控且不协调的写入会制造出难以恢复的混沌与溢出**。

LSTM 的核心挑战：失控且不协调的写入

失控且不协调的写入，尤其是在训练初期写入完全随机时，会制造出混乱的状态，导致结果变差，并且很难从这种混乱中恢复。

Hochreiter 和 Schmidhuber 意识到了这个问题，并将其拆解为几个子问题，他们称之为：

- **输入权重冲突 (input weight conflict)**

- 输出权重冲突 (output weight conflict)
- 滥用问题 (abuse problem)
- 内部状态漂移 (internal state drift)

LSTM 架构正是为了解决这些问题而精心设计的，其核心思想始于“选择性”。

用选择性控制与协调写入 (Using selectivity to control and coordinate writing)

根据早期 LSTM 文献，克服核心挑战、稳定状态的关键，是在三件事上保持**选择性**：

1. **写什么** (选择性写入)
2. **读什么** (要写先得读)
3. **忘什么** (过时信息是干扰，应丢弃。)

比如COVID-19的疫情信息会表明我们需要囤积医疗物资，注意呼吸道卫生，但显然继续采纳此类信息在今天会影响我们的生活，造成不必要的浪费。我们需要抛弃此信息。

状态混乱的原因之一： 基础 RNN 会因为**写入状态的每一位**而变得chaotic。这就像我在同一张纸上写满东西，写满换一张，最后桌上堆满杂乱无章的纸。

Hochreiter & Schmidhuber 称之为**输入权重冲突**：如果每个单元每一时间步都被所有单元写入，会积累大量无用信息，原有状态失效。RNN 必须学会用某些单元**抵消写入**、保护状态，导致它的学习困难。

第一类选择性：选择性写入

现实中要让记录有用，必须**选择性写**。上课只记重点，绝不把新笔记写在旧笔记上。RNN 单元也需要**选择性写入机制**。

状态混乱的原因之二： 它是第一个原因的另一面：**基础循环神经网络 (RNN)** 每执行一次**写入操作**，都会读取状态中的每一个元素。举一个温和的例子：如果我在国家公园度假时写一篇关于**长短期记忆网络 (LSTMs)** 背后直觉的博客文章，而此时公园里有一只野生熊在游荡，我可能会在博客中加入一些我读到的关于熊类安全的内容。这只是一件小事，混乱程度也比较轻微，但试想一下，如果我把所有接触到的东西都**写出来**，这篇文章会变成什么样子.....

霍赫赖特 (Hochreiter) 和施密德胡贝尔 (Schmidhuber) 将这种现象描述为“**输出权重冲突**”：如果**无关单元**在每个时间步都被所有其他单元读取，就会产生大量潜在的**无关信息**。因此，RNN 必须学会利用部分单元来**抵消无关信息**，这会导致学习过程变得困难。

注意读与写的区别：

读取的信息会被汇总起来，并通过**非线性函数**压缩，而写入的信息是绝对的。因此，读取决策的影响**范围广但程度浅**，而写入决策的影响**范围窄但程度深**。

概括一下：

- 不读某个单元 → 它无法影响我们状态的任何元素，
- 读了某个单元 → 影响**广而浅**
- 不写某个单元 → 只影响状态中的那一个特定元素，
- 写了某个单元 → 写入的信息是绝对的，影响**窄而深**

第二类选择性：选择性读取

为了在现实世界中过得好，我们需要通过对阅读或消费的内容进行筛选，来应用最相关的知识。为了让我们的 RNN（循环神经网络）单元也能做到这一点，它们需要一种选择性读取的机制。

第三种选择性形式与我们如何处理不再需要的信息有关。这关乎我们要如何处理那些不再需要的信息。那些旧的论文笔记会被**扔掉**。否则，即便我在写笔记时已经很挑剔（只记重点），最终我还是会被堆积如山的纸张淹没。我 Dropbox（云盘）里不用的文件会被**覆盖**。否则，即便我在创建文件时很**谨慎地筛选过了**，我的存储空间迟早也会**耗尽**。

这种**直觉**并非在原始的LSTM论文中提出，这导致原始LSTM模型在处理涉及**长序列**的简单任务时会遇到困难。相反，它是由[Gers等人（2000年）](#)提出的。根据Gers等人的研究，在某些情况下，原始LSTM模型的**状态**会无限增长，最终导致网络**崩溃**。换句话说，原始LSTM存在**信息过载**的问题。

第三类选择性：选择性遗忘

现实中我们记不住无限多东西；为给新信息腾空间，必须**选择性遗忘最不重要的旧信息**。

这个直觉并非来自最初 LSTM 论文（原始 LSTM 在长序列上表现不佳），而是 [Gers et al. \(2000\)](#) 引入。他们发现原始 LSTM 状态会无限增长，最终网络崩溃，即**信息过载**。

至此，推导 LSTM 只剩两步：

1. 找到**选择性机制**
2. 把**组件拼接起来**

门：选择性的实现机制（Gates as a mechanism for selectivity）

选择性读取、写入与遗忘，是指对状态向量的每个元素，分别做出独立的读取、写入和遗忘决策。我们将借助与**状态维度相同**的读取向量、写入向量和遗忘向量来实现这些决策：向量中每个元素的值都在 0 到 1 之间，用来指定我们对状态中对应元素执行读取、写入或遗忘操作的“比例”。

需要注意的是：虽然把读取、写入和遗忘看作**二元决策**（非 0 即 1）会更符合直觉，但为了能通过梯度下降来训练模型，我们必须用**可微函数**来实现这些决策。逻辑 sigmoid 函数是一个非常自然的选择——它本身是可微的，并且输出值恰好落在 0 到 1 之间。

我们将这些读、写和遗忘向量称为**门**，并可以使用最简单的函数来计算它们——就像我们在普通循环神经网络（Vanilla RNN）中所做的那样：单层神经网络。在时间步 t 下，我们的三个门分别表示为：

- i_t ：输入门（控制写入input）
- o_t ：输出门（控制读取output）
- f_t ：遗忘门（控制保留/遗忘）

从这些名称中，我们立刻能注意到，LSTM 中有两处概念是“反过来”的：

- 诚然这有点像“先有鸡还是先有蛋”的问题，
- 但通常我们会认为流程是**先读取read，再写入write**。事实上，循环神经网络（RNN）单元的定义也强烈暗示了这种顺序——我们需要先读取上一时刻的状态，才能写入新状态；即便初始状态是空的，我们也需要先从它读取。而**输入门**和**输出门**的名称暗示了相反的时序关系，这正是 LSTM 采用的方式。我们会看到，这会让架构变得更复杂。
- **遗忘门**（forget gate）的名字本意为“遗忘”，但它实际上的作用更像一个**记忆门**，
 - 值越大、记得越多。

例如，遗忘门向量中的值为 1 时，代表“**记住所有信息**”，而非“**遗忘所有信息**”。

这在实际功能上并无差别，但很容易造成理解上的混淆。

数学定义（注意它们的相似性）：

$$\begin{aligned}i_t &= \sigma(W_i s_{t-1} + U_i x_t + b_i) \\o_t &= \sigma(W_o s_{t-1} + U_o x_t + b_o) \\f_t &= \sigma(W_f s_{t-1} + U_f x_t + b_f)\end{aligned}\tag{2}$$

门也可以用更复杂的函数计算，比如近年有效的“乘法积分”([Wu et al., 2016](#))。

拼接门，推导出原型 LSTM（Gluing gates together to derive a prototype LSTM）

如果没有写入门，选择性读取意味着：在产生下一次写入前，应该先用读取门处理上一状态。LSTM 核心原则说写入是**增量**的，所以我们算的是 Δs_t ，不是 s_t 。我们把这个待写入量称为**候选状态**，记作 $\tilde{s}_t$ 。

计算 $\tilde{s}_t$ 的方式和普通 RNN 算状态一样，只是不直接用原始上一状态 s_{t-1} ，而是先用输出门（读取门）逐元素相乘，得到门控后状态：

$$\tilde{s}_t = \phi(W(o_t \odot s_{t-1}) + Ux_t + b)$$

$\odot$ 表示逐元素相乘。 o_t 是RNN意义的read gate，也是LSTM意义的 **output gate**。

$\tilde{s}_t$ 只是候选，因为要选择性地写入并且已经有了一个write gate（LSTM的**input**，RNN的write），所以用**输入门**逐元素调制，得到真正写入： $i_t \odot \tilde{s}_t$

最后一步：把写入加到上一状态。但选择性遗忘要求我们先处理旧信息。所以在加新内容前，先用**遗忘门**逐元素乘上一状态（遗忘门实际是**保留门**：1 表示全部保留，0 表示全部遗忘）。

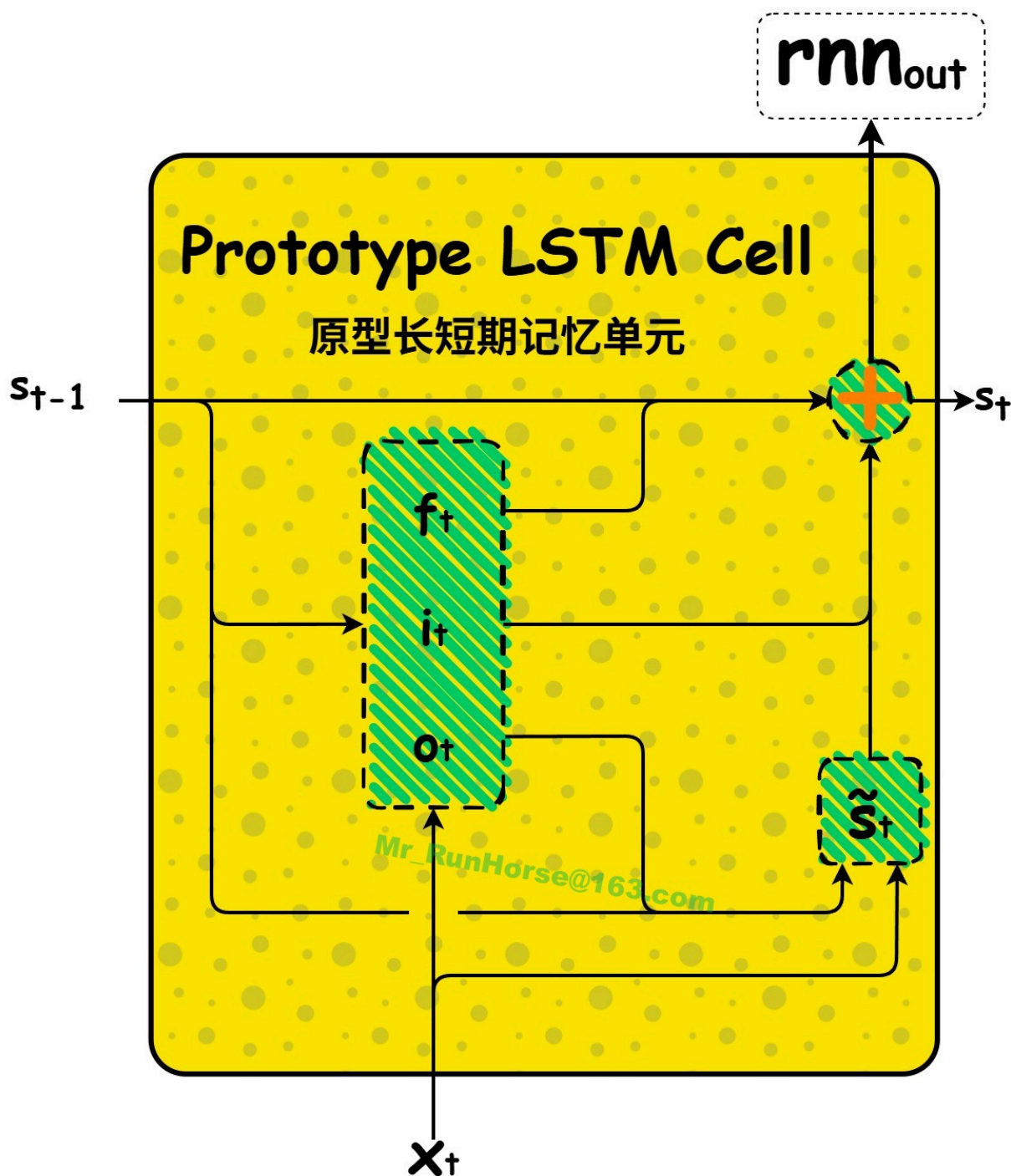
最终**原型 LSTM** 公式：

$$s_t = f_t \odot s_{t-1} + i_t \odot \tilde{s}_t$$

把所有公式合起来，就是原型 LSTM 单元完整定义（ s_t 同时是外部输出）：

原型 LSTM

$$\begin{aligned} i_t &= \sigma(W_i s_{t-1} + U_i x_t + b_i) \\ o_t &= \sigma(W_o s_{t-1} + U_o x_t + b_o) \\ f_t &= \sigma(W_f s_{t-1} + U_f x_t + b_f) \\ \tilde{s}_t &= \phi(W(o_t \odot s_{t-1}) + Ux_t + b) \\ s_t &= f_t \odot s_{t-1} + i_t \odot \tilde{s}_t \end{aligned} \tag{3}$$



理论上，这个原型是可以正常工作的；如果真的能这样，它的设计会非常简洁优雅。但在实际应用中，这些选择性控制的措施（通常）不足以克服 LSTM 的核心挑战：训练初期，**选择性遗忘与选择性写入之间缺乏协调**，这会导致状态值迅速膨胀并变得混乱。此外，由于状态的取值范围理论上是无界的，门控和候选写入值往往会进入饱和区，给训练带来严重问题。

Hochreiter 和 Schmidhuber (1997) 最早观察到了这一现象，并将其命名为“**内部状态漂移 (internal state drift)**”，他们的解释是：“如果（写入操作）的值大多为正或大多为负，那么内部状态就会随着时间推移逐渐偏离初始值”。

事实证明，这个问题的影响极为严重，以至于我们上面设计的原型模型在实际中往往会失效——即便使用极小的初始学习率、精心调整偏置项初始化，也难以避免。[Greff et al. \(2015\)](#)的研究给

出了最清晰的实证证明：该论文对比了 8 种 LSTM 变体的实际效果，其中表现最差、最常出现无法收敛问题的变体，其结构与我们的原型几乎完全一致。

解决办法：**给状态加约束，防止爆炸**。不同约束方式引出不同 LSTM 模型。

三种可用模型：归一化原型、GRU、伪 LSTM

我们在原型 LSTM 中采用的选择性控制机制，不足以克服 LSTM 面临的核心挑战。具体来说，用于计算门控和候选写入值的状态，会出现**无界增长**的问题。

下面三种方法都通过**约束状态**得到可用 LSTM：

1. 归一化原型：用归一化的软约束 (The normalized prototype: a soft bound via normalization)

我们可以通过**状态归一化**施加软约束。

一种在初步测试中效果不错的简单做法：将状态 s_t 除以 $\sqrt{\text{Var}(s_t) + 1}$ ，

+1：防止训练刚开始时，所有 s_t 都是**初始值 0**，导致**方差Var**为 0

结果：状态 s_t 除以 $\sqrt{\text{Var}(s_t) + 1}$ **分母为 0 或者接近 0**，出现数值爆炸

为什么不是+2 +3 等等大于1的数？

结果：某次Var为0，输出状态**会被缩小**

也可以先减去状态均值，再做方差Var归一化，但在初步测试中，这种做法并没有带来明显增益。

后续还可以仿照**层归一化**的思路，引入缩放因子与平移因子来提升模型表达能力；

但这样一来，模型就进入了**层归一化 LSTM** 的范畴，后续还需要和其他层归一化 LSTM 变体做对比。

无论哪种方式，该方法都能对状态施加**软边界约束**。

在我的初步实验中，它的表现略优于标准 LSTM（也包括下文推导的伪 LSTM 结构）。

2. GRU：通过写-忘耦合做硬约束/覆盖 (a hard bound via write-forget coupling, or overwriting)

可以通过**显式关联写入与遗忘机制**，给细胞状态施加**硬约束**，同时让二者相互协调；

换言之，不再分开执行独立的选择性写入、选择性遗忘，而是牺牲一部分模型表达能力，改用**选择性覆写**：

令遗忘门= $1 - \text{写入门}$ ，从而满足如下关系：

$$s_t = (1 - i_t) \odot s_{t-1} + i_t \odot \tilde{s}_t$$

这样设计可行，是因为它会将状态 s_t 转化为 s_{t-1} 与候选状态 $\tilde{s}_t$

逐元素的加权平均；只要 s_{t-1} 和 $\tilde{s}_t$ 自身有界， s_t 就天然具备有界性。

若选用激活函数 $\phi = \tanh$ （输出被约束在 $(-1, 1)$ 区间内），即可满足上述有界条件。

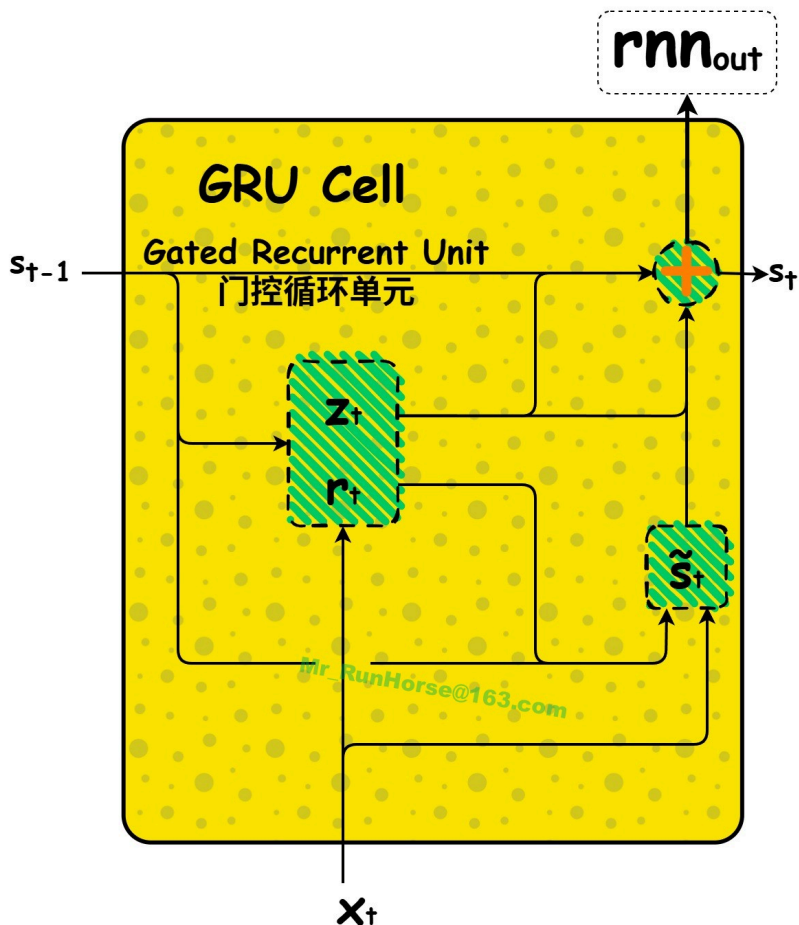
至此我们就推导出了**门控循环单元（GRU）**。为契合学术界通用的 GRU 术语，我们将覆写门改称**更新门**，记为 z_t 。

需要注意：虽名为**更新门**，实际作用等价于**不更新门**——它用来指定**保留多少比例的历史旧状态**、不被新信息覆写。因此更新门 z_t 等价于原型 LSTM 中的**遗忘门** f_t ，而写入门由 $1 - z_t$ 计算得到。

另外有意思的是：提出 GRU 的作者，把其中的**读取门**命名为**重置门**（好歹我们还能用符号 r_t 来表示它！）。

GRU

$$\begin{aligned} r_t &= \sigma(W_r s_{t-1} + U_r x_t + b_r) \\ z_t &= \sigma(W_z s_{t-1} + U_z x_t + b_z) \\ \tilde{s}_t &= \phi(W(r_t \odot s_{t-1}) + U x_t + b) \\ s_t &= z_t \odot s_{t-1} + (1 - z_t) \odot \tilde{s}_t \end{aligned} \tag{4}$$



这就是 [Cho et al. \(2014\)](#) 首次提出的 **GRU 单元**。相信你能认同，本文对 GRU 的推导，每一步都具备合理的动机。

整个逻辑推导过程中没有任何一处生硬突兀的人为设定，这一点和后面推导 LSTM 的过程截然不同（LSTM 会存在多处强行设定的环节）。与部分文献观点相反（例如 [Jozefowicz et al. \(2015\)](#) 所述：“GRU 作为 LSTM 的替代模型，同样难以从逻辑上合理解释其设计”），我们可以清晰看出：**GRU 是一种推导逻辑非常自然的网络结构。**

3. 伪 LSTM：用非线性压缩做硬约束

我们走向完整 LSTM 的倒数第二步：用**压缩函数**（Sigmoid/Tanh）约束状态。关键是：**不能直接压缩状态本身**（否则会造成信息畸变，违背 LSTM 核心原则）。正确做法：**只有在增量写入以外的场景，才把状态压缩**。这样门和候选状态不会饱和，同时保持良好梯度流动。

到目前为止，外部输出 = 状态。但在伪 LSTM 里，只有增量写入时不压缩状态，其他时候都压缩。因此单元输出与状态分离。

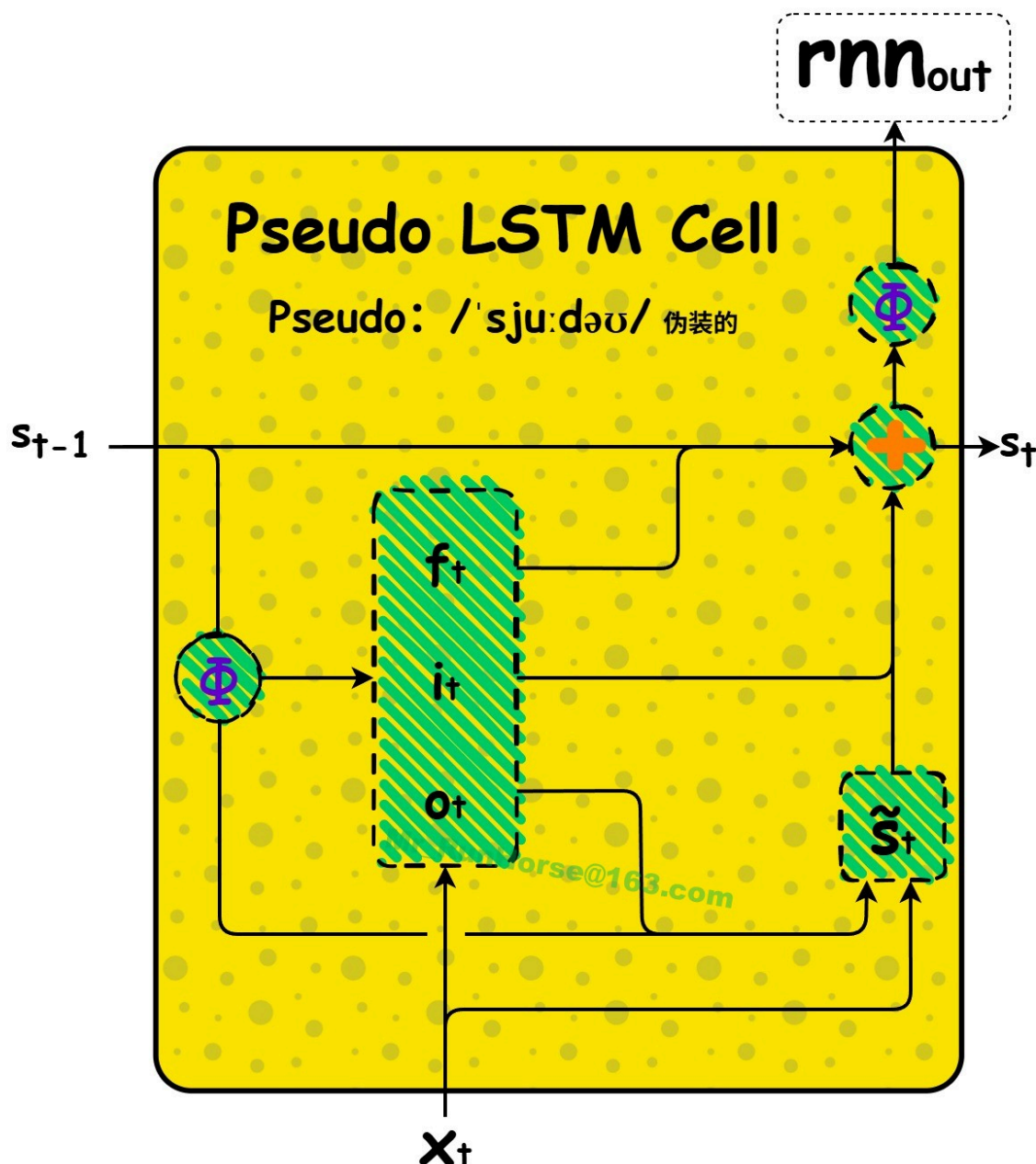
修改很直接，用 ϕ 表示压缩函数（通常和候选状态的非线性一样用 tanh）：

伪 LSTM

$$\begin{aligned}
 i_t &= \sigma(W_i \phi(s_{t-1}) + U_i x_t + b_i) \\
 o_t &= \sigma(W_o \phi(s_{t-1}) + U_o x_t + b_o) \\
 f_t &= \sigma(W_f \phi(s_{t-1}) + U_f x_t + b_f)
 \end{aligned}$$

$$\begin{aligned}
 \tilde{s}_t &= \phi(W(o_t \odot \phi(s_{t-1})) + Ux_t + b) \\
 s_t &= f_t \odot s_{t-1} + i_t \odot \tilde{s}_t
 \end{aligned}
 \tag{5}$$

$$\text{rnn}_{\text{out}} = \phi(s_t)$$



伪 LSTM 几乎就是 LSTM，只是顺序反了。从这里可以清楚看到：GRU 与 LSTM 的核心动机差异，只在于状态约束方式。伪 LSTM 相对标准 LSTM 有一些优势。

标准 LSTM：先丢旧信息（遗忘），再补新信息（输入门写入）

伪 LSTM：先把新信息写进去，再决定丢多少旧信息

- 比较

原型 LSTM 的核心更新公式

$$s_t = \underbrace{f_t \odot s_{t-1}}_{\text{先遗忘旧状态}} + \underbrace{i_t \odot \tilde{s}_t}_{\text{再写入新信息}}$$

- f_t (遗忘门) 直接作用于 s_{t-1}
- $\tilde{s}_t$ 由 x_t 和 $\phi(s_{t-1})$ 共同计算，但不经过门控作用

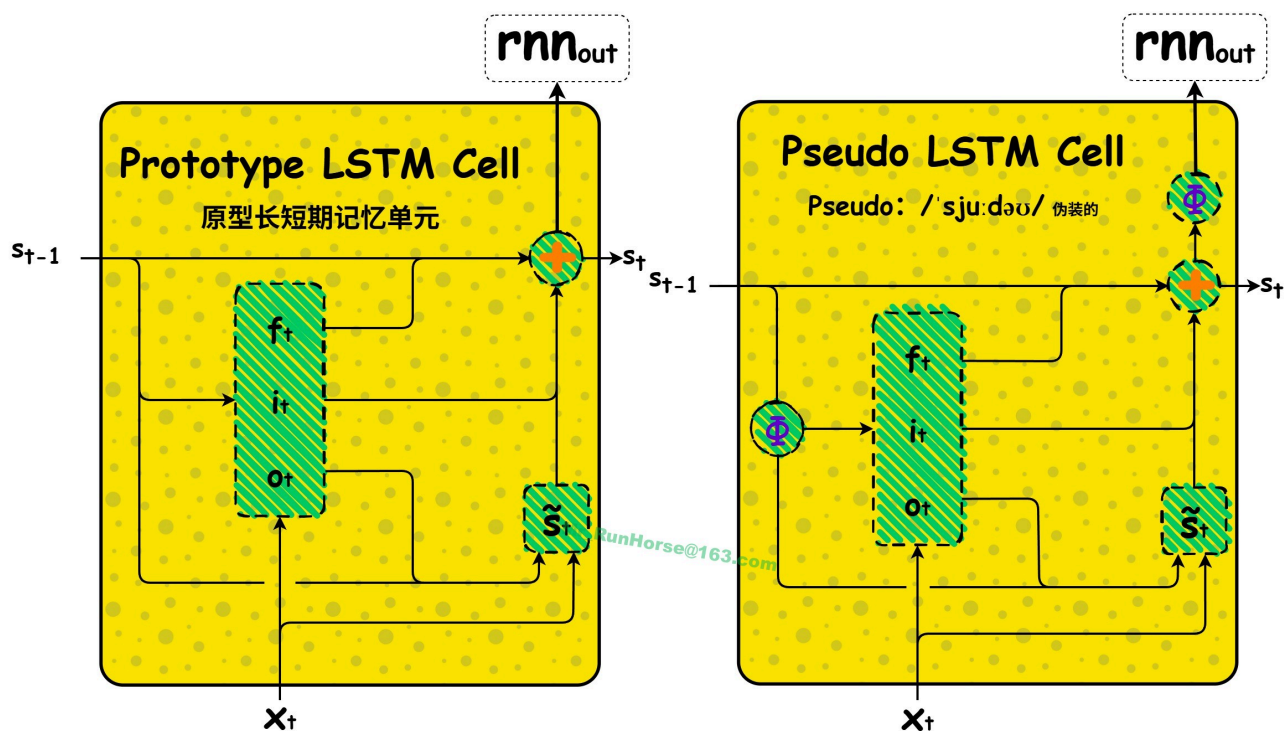
伪 LSTM 的核心更新公式

$$s_t = \underbrace{i_t \odot s_{t-1}}_{\text{先写入保留的旧状态}} + \underbrace{f_t \odot \tilde{s}_t}_{\text{再筛选新信息}}$$

- i_t (输入门) 直接作用于 s_{t-1}
- $\tilde{s}_t$ 由 x_t 和 $o_t \odot \phi(s_{t-1})$ 共同计算，输出门 o_t 参与新信息生成

整体比较

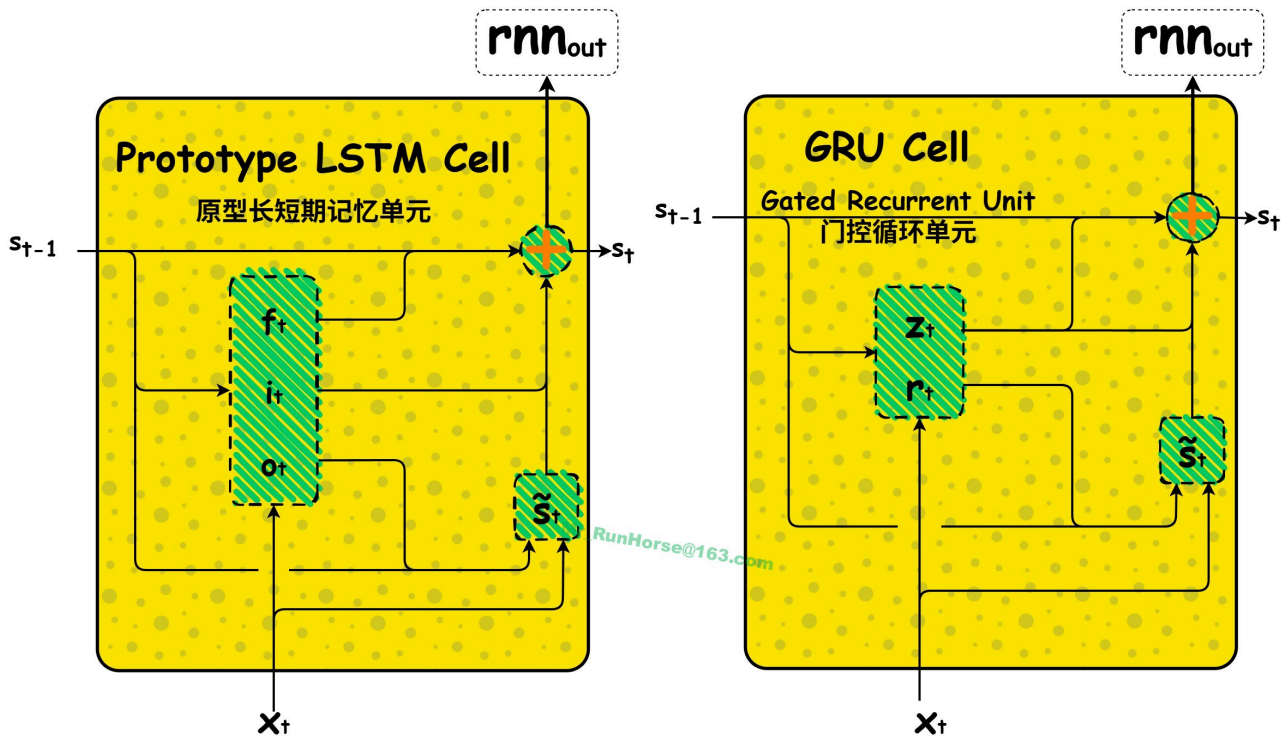
原型LSTM：伪LSTM



原型 LSTM	伪 LSTM
$i_t = \sigma(W_i s_{t-1} + U_i x_t + b_i)$ $o_t = \sigma(W_o s_{t-1} + U_o x_t + b_o)$ $f_t = \sigma(W_f s_{t-1} + U_f x_t + b_f)$ $\tilde{s}_t = \phi(W(o_t \odot s_{t-1}) + Ux_t + b)$ $s_t = f_t \odot s_{t-1} + i_t \odot \tilde{s}_t$ $rnn_{out} = s_t \quad (\text{原型隐式输出})$	$i_t = \sigma(W_i \phi(s_{t-1}) + U_i x_t + b_i)$ $o_t = \sigma(W_o \phi(s_{t-1}) + U_o x_t + b_o)$ $f_t = \sigma(W_f \phi(s_{t-1}) + U_f x_t + b_f)$ $\tilde{s}_t = \phi(W(o_t \odot \phi(s_{t-1})) + Ux_t + b)$ $s_t = f_t \odot s_{t-1} + i_t \odot \tilde{s}_t$ $rnn_{out} = \phi(s_t) \quad (\text{伪LSTM显式输出})$

注：归一化原型LSTM用状态 s_t 除以 $\sqrt{\text{Var}(s_t) + 1}$ 作为 rnn_{out}

原型LSTM：GRU



原型 LSTM	GRU
$i_t = \sigma(W_i s_{t-1} + U_i x_t + b_i)$ $o_t = \sigma(W_o s_{t-1} + U_o x_t + b_o)$ $f_t = \sigma(W_f s_{t-1} + U_f x_t + b_f)$ $\tilde{s}_t = \phi(W(o_t \odot s_{t-1}) + Ux_t + b)$ $s_t = f_t \odot s_{t-1} + i_t \odot \tilde{s}_t$ $rnn_{out} = s_t$	$r_t = \sigma(W_r s_{t-1} + U_r x_t + b_r)$ $z_t = \sigma(W_z s_{t-1} + U_z x_t + b_z)$ $\tilde{s}_t = \phi(W(r_t \odot s_{t-1}) + Ux_t + b)$ $s_t = z_t \odot s_{t-1} + (1 - z_t) \odot \tilde{s}_t$ $rnn_{out} = s_t$

注：归一化原型LSTM用状态 s_t 除以 $\sqrt{\text{Var}(s_t) + 1}$ 作为 rnn_{out}

伪 LSTM 算不算 LSTM?

不算标准原版 LSTM

- 标准 LSTM 固定结构：遗忘门控旧状态、输入门控新候选状态；
- 伪 LSTM 是把门控作用对象、数据流顺序完全倒置：用输入门控旧状态、遗忘门控新候选状态；
- 结构同源、参数形式几乎一样，但门控定义、信息流逻辑反了，属于 LSTM 变体 / 反向推导版，不是正统 LSTM。

机器学习实际工程里会用伪 LSTM 吗？

工业界、主流项目基本不用

1. 教科书、开源框架（PyTorch/TensorFlow）只实现**标准 LSTM、GRU**，没有伪 LSTM 原生实现；
2. 伪 LSTM 更多出现在**理论推导、教学科普**里：用来演示：
 1. LSTM 为什么设计成现在这个顺序
 2. 状态漂移、有界性怎么来的
 3. GRU 是怎么从这套逻辑推出来的
3. 少数论文做**变体对比实验**会拿它当对照组，但**不会拿来落地业务、时序预测、语音、NLP 任务**。

一句话总结

伪 LSTM：理论教学推导，看懂 LSTM/GRU 底层逻辑用，实战工程没人用，不属于标准 LSTM。

三、推导 LSTM（Deriving the LSTM）

文献中有多种 LSTM 变体，但差异对我们不重要。它们和**伪 LSTM**有一个**关键不同**：

真实 LSTM 把读取操作放在写入操作之后。

LSTM 差异 1（LSTM 的小插曲）

读操作发生在写操作之后，这迫使 **LSTM** 在时间步之间传递一个**影子状态**。

如果你阅读了霍赫赖特（Hochreiter）和施密德胡伯（Schmidhuber）（1997）的论文，就会发现，他们所设想的“状态”（正如我们迄今所使用的“状态”概念）与 RNN 单元的其余部分是相互独立的。¹² 霍赫赖特和施密德胡伯将状态视为一种**记忆单元（memory cell）**，这种单元具有**恒定误差**

(**constant error**) (因为在没有读写操作的情况下，它会持续传递状态，并在反向传播过程中保持恒定的梯度)。或许正因如此，他们将状态视为一个独立的存储单元，从而认为操作顺序应为先写入（输入），再读取（输出）。事实上，大多数LSTM的示意图——包括Hochreiter和Schmidhuber (1997) 以及 [Graves \(2013\)](#) 中的那些——都容易让人困惑，因为它们聚焦于这个“存储单元”，而非整个LSTM单元本身。¹³ 这里没有列出示例，以免分散对原始理解的注意力。

这一设计确实是个小插曲，而且并非因为它让事情变得更复杂（实际上它的确让事情更复杂了）。核心问题在于：我们最终使用的读取门控，是在 $t-1$ 时刻计算完成的，基于 $t-2$ 时刻的影子状态与 $t-1$ 时刻的输入，用来门控筛选 t 时刻使用的状态信息。这就像**基于昨日新闻进行的日内交易**，门控信号存在天然的滞后性。

为贴合LSTM通用符号规范，我们将主状态 s_t 重命名为 c_t （c代表单元或恒定误差）；同步将候选写入状态修改为 $\tilde{c}_t$ ；同时引入独立的**影子状态** h_t （h代表隐藏状态），维度与主状态完全一致。 h_{t-1} 对应原型LSTM中经过门控的前序状态 $o_{t-1} \odot s_{t-1}$ ，额外经过非线性变换压缩，约束计算候选写入时的数值范围。因此，LSTM在时间步 t 接收的前序状态，是一组强关联的向量元组： (c_{t-1}, h_{t-1}) ，满足约束关系 $h_{t-1} = o_{t-1} \odot \phi(c_{t-1})$ 。

这一特殊设定带来了 h_{t-1} ，也直接催生了伪LSTM与标准LSTM的另外**两项**核心差异：

1. 首先，标准 LSTM 并没有使用（经过压缩的）无门控前一时刻状态 $\phi(c_{t-1})$ 来计算门控，而是使用了经过**读取门控**作用的 $h_{t-1} = o_{t-1} \odot \phi(c_{t-1})$ ，且这个读取门控还是**上一时刻的旧值**。

LSTM 差异 2

标准 LSTM 计算门控时，使用的是**经过门控的影子状态**： $h_{t-1} = o_{t-1} \odot \phi(c_{t-1})$ ，而非直接使用经过压缩的主状态： $\phi(c_{t-1})$ 。

2. 其次，标准 LSTM 也不直接将（经过压缩的）无门控状态 $\phi(c_t)$ 作为外部输出，而是使用**经过读取门控作用后的状态**： $h_t = o_t \odot \phi(c_t)$

LSTM 差异 3

标准LSTM使用经过当前时刻读取门控筛选后的**门控影子状态** $h_t = o_t \odot \phi(c_t)$ ，不会将未经门控的**压缩状态** $\phi(c_t)$ 作为对外输出。

尽管我们可以理解这些差异是如何产生的——这是由LSTM真实状态的“记忆单元”视角所导致的——但至少差异1和差异3缺少严谨的理论动机。也正因如此，我并不认同 [Jozefowicz et al. \(2015\)](#) 关于 GRU“难以解释”的观点，但完全认同：LSTM存在部分结构组件，**设计用途无法直观、快速地被理解**。

基于以上三项核心差异，我们反向重写伪LSTM，即可得到真正的LSTM。它在每一步接收两个前序状态量 c_{t-1} 和 h_{t-1} ，生成并传递给下一时刻两个状态量 c_t 和 h_t 。我们得到的这个LSTM非常“标准”：这就是你在Tensorflow中能找到的“BasicLSTMCell”实现版本。

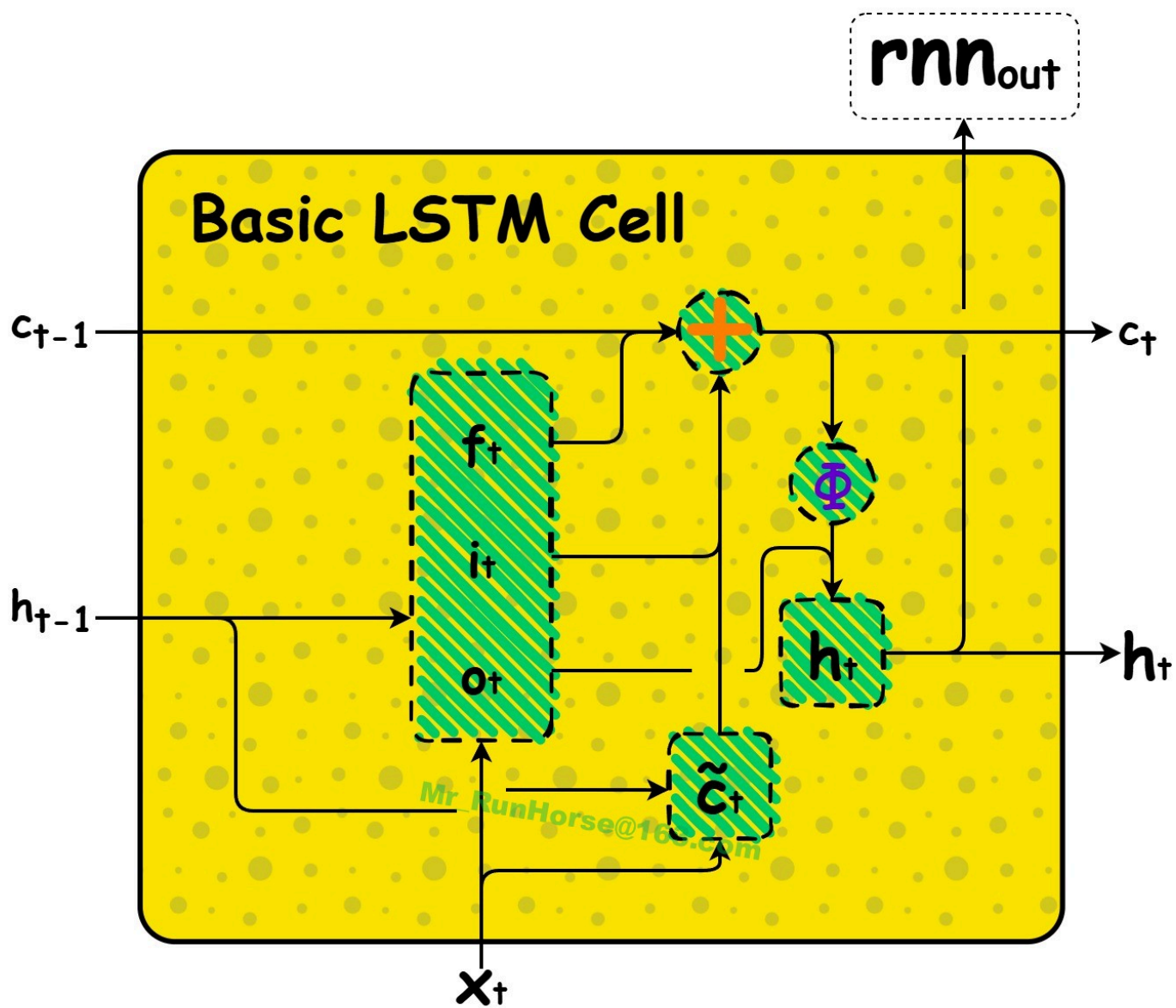
标准LSTM完整公式

$$\begin{aligned}i_t &= \sigma(W_i h_{t-1} + U_i x_t + b_i) \\o_t &= \sigma(W_o h_{t-1} + U_o x_t + b_o) \\f_t &= \sigma(W_f h_{t-1} + U_f x_t + b_f)\end{aligned}$$

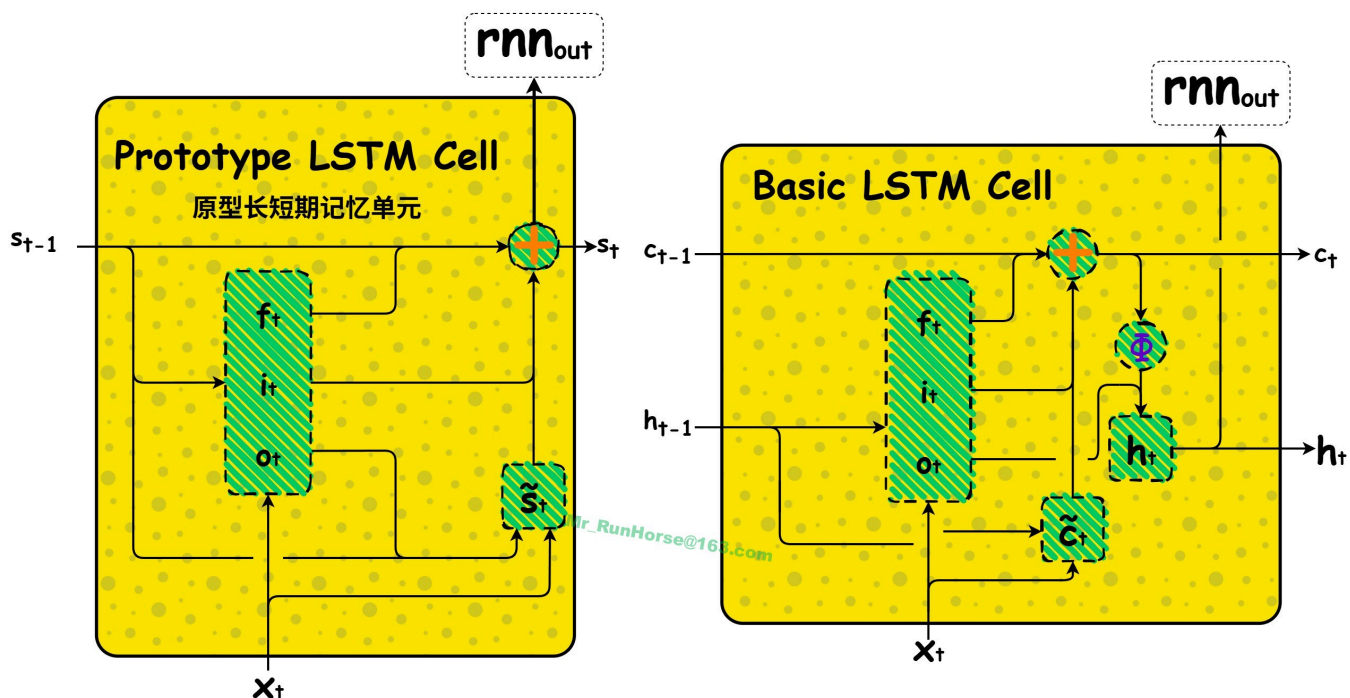
$$\begin{aligned}\tilde{c}_t &= \phi(W h_{t-1} + U x_t + b) \\c_t &= f_t \odot c_{t-1} + i_t \odot \tilde{c}_t\end{aligned}\tag{6}$$

$$h_t = o_t \odot \phi(c_t)$$

$$\text{rnn}_{\text{out}} = h_t$$



原型LSTM：标准LSTM



原型 LSTM	标准 LSTM
$i_t = \sigma(W_i s_{t-1} + U_i x_t + b_i)$ $o_t = \sigma(W_o s_{t-1} + U_o x_t + b_o)$ $f_t = \sigma(W_f s_{t-1} + U_f x_t + b_f)$ $\tilde{s}_t = \phi(W(o_t \odot s_{t-1}) + Ux_t + b)$ $s_t = f_t \odot s_{t-1} + i_t \odot \tilde{s}_t$ $rnn_{out} = s_t$	$i_t = \sigma(W_i h_{t-1} + U_i x_t + b_i)$ $o_t = \sigma(W_o h_{t-1} + U_o x_t + b_o)$ $f_t = \sigma(W_f h_{t-1} + U_f x_t + b_f)$ $\tilde{c}_t = \phi(Wh_{t-1} + Ux_t + b)$ $c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$ $h_t = o_t \odot \phi(c_t)$ $rnn_{out} = h_t$

虽然门控之间不共享权重，但同一个门控在不同时间步之间的权重是完全共享的。

The LSTM with peepholes

到此为止，以后继续